

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Dumbbell Matching (33 points)

You belong to a gym which has two sets of dumbbells A and B , each of which has n dumbbells. You know that there is a correspondence between the sets: for every dumbbell in set A there is exactly one dumbbell in set B that has the same weight, and similarly for every dumbbell in set B there is exactly one dumbbell in set A that has the same weight. You want to perform exercises that require two dumbbells of the same weight. So you want to pair up the dumbbells by weight, i.e., for every dumbbell you want to know which dumbbell from the other set has the exact same weight.

Unfortunately the dumbbells are unsorted and unlabeled, and you can't tell their weights by looking at them. The only way to compare two dumbbells is to pick them both up simultaneously (one in each hand) and perform a curl. By comparing the strain on your arms, you can tell whether the two dumbbells are the same weight, and if not, which one is heavier. Even more unfortunately, the owner of the gym has a rule that two dumbbells from the same set cannot be used at the same time. So you can compare a dumbbell from set A to a dumbbell from set B , but cannot compare two dumbbells from the same set.

Prove that any deterministic algorithm which solves the problem must make at least $\Omega(n \log n)$ comparisons.

Answer: We will use a similar idea to the one we used in class when proving the $\Omega(n \log n)$ lower bound for comparison-based sorting algorithms. We are trying to match the weight ordering of two sets of dumbbells, so we must perform comparisons to find the matches. Finally, as we do not have a range of weights or a means of determining the value of any weight, we cannot use a non-comparison based algorithm which may order the weights.

Suppose that we first fixed the ordering of the set A , meaning that we find ordering π for set B so that $weight(a_i) = weight(b_{\pi(i)})$. Therefore, the question shifts to sorting algorithm for set B where the sorting is done with respect to the ordering of set A . This results in a decision tree method. In this case, there are three children for each node. Suppose you start with the comparison $a_1 : b_1$, which spouts one branch per element of set $\{>, =, <\}$, which contains the possible relationships between a_1 and b_1 . Since $|A| = |B| = n$, then this means there are $n!$ distinct orderings/permutations for set B . Hence, our decision tree would have at least $n!$ leaves. The maximum number of comparisons you would need to sort set B is actually the depth of this decision tree. Thus, the following describes the number of decisions (d) that must be made by any algorithm to produce ordering π for set B . Therefore, we can build the following inequality:

$$n! \leq \# \text{leaves} \leq 3^d$$

This is equivalent to:

$$\log_3(n!) \leq d$$

Since the number of decisions that must be made is greater than or equal to $\log_3(n!)$, this must be the lower bound for the number of comparisons. Using the same logic as in class to prove a minimum $\log(n!)$ comparisons means that any sorting algorithm has lower bound $\Omega(n \log n)$, taken in account with the fact that changing the base of a log changes it to a constant value, we can represent $\log_3(n!)$ as $\Omega(n \log n)$.

Therefore, any deterministic algorithm solving this problem must make at least $\Omega(n \log n)$ comparisons.

2 Group Sorting (33 points)

In the last homework you designed an algorithm for k -group-sorting in time $O(n \log k)$. Prove a matching lower bound in the comparison model. That is, prove that any algorithm for k -group-sorting in the comparison model must make at least $\Omega(n \log k)$ comparisons in the worst case.

Answer: Let us consider the number of permutations of array A to determine the number of comparisons needed for k -group sorting. We assume A has n elements, and we are trying to sort it into k groups. On the whole, since there are n elements, there are $n!$ permutations of A . Now, let us calculate the number of permutations of A for which $n!$ is k -group sorted. Since the array is broken up in k groups, each group would have $\frac{n}{k}$ elements in it. Therefore, within each group, there are $\frac{n}{k}!$ permutations. Each of the k groups independently has this $\frac{n}{k}!$ number of permutations. Therefore, the number of permutations of A such that A is k -group sorted would be $(\frac{n}{k}!)^k$.

Since there are $n!$ total permutations of A , and there are $(\frac{n}{k}!)^k$ k -group sorted permutations of A , we can determine the number of different outcomes after any algorithm is applied to be their quotient, that is, there are $\frac{n!}{((n/k)!)^k}$ outcomes.

Each comparison of any two values x and y in A has 3 possible outcomes.

- (1) $x \leq y$
- (2) $x > y$

Therefore, an algorithm that makes d comparisons would have at most 2^d leaves, so

$$\frac{n!}{((n/k)!)^k} \leq \# \text{leaves} \leq 2^d$$

This can be written as:

$$d \leq \log\left(\frac{n!}{((n/k)!)^k}\right)$$

We can simplify this further by finding a lower bound for the numerator and an upper bound for the denominator to prove a lower bound for the algorithm as a whole:

$$\left(\frac{n!}{((n/k)!)^k}\right) \geq \frac{(n/e)^n}{(n/k)^{(n/k)k}} = \left(\frac{n}{e(n/k)}\right)^n = (k/e)^n$$

This means that $d \geq n \log(k/e)$, so $d = \Omega(n \log k)$. Thus, we have proven a minimum of $\Omega(n \log k)$ comparisons for any algorithm that completes the k -group sorting task.

3 Sorting Different-Length Items (34 points)

We saw in class some fast sorting algorithms where we assumed that the elements were integers with the same number of digits. In this problem we change those assumptions slightly.

Suppose now that we are given an array of strings (over some finite alphabet, say the letters **a-z**). Each string can have a different number of characters, but the total number of characters in all the strings (i.e., the sum over the strings of the length of the string) is equal to n . Show to sort the strings lexicographically in $O(n)$ time. Here lexicographic order is the standard alphabetic order, so for example **a** < **ab** < **b**.

Answer: We will approach this question rather a reverse manner. We will sort the words by their length. Then we will sort them by their last character first and then their second-last, and so on up to their first character. An example would be: Say we are given Array A , containing elements $[b, cd, a, cab, ab, bcd]$. First, we would use counting sort to inspect each string by length and sort by length, which is later yesed to maintain the rule that $a < aa < ab < b$. Thus, after this step, A would be reconstructed as would have $[b, a, cd, ab, cab, bcd]$. Let us assign k to mean the greatest string length. In the cast of A , $k = 3$. We first examine the k -character strings, using counting sort to sort by the k th character, which would yield $A = [b, a, cd, ab, cab, bcd]$. Then, we look at the $k - 1$ th character, yielding $A = [b, a, cab, ab, bcd, cd]$. This process continues for any array until the algorithm examines the first character of every element of the array. For A , this yields $A = [a, ab, b, bcd, cab, cd]$.

Correctness Proof: First let us introduce some notation. We define A as any array made up of strings, where the total number of characters across all the strings is n . We define L is the number of characters in the longest word.

It is obvious that we can always sort the words by their length, using a counting sort, since each string has at most n characters. We can create buckets from 1 to n and reconstruct the array in sorted length.

We will use induction to prove this algorithm's correctness. I will call this subsorting algorithm M , and thus $M(i)$ is the counting sort applied to the i th character of each string in A . Instead of traditional basic induction, we will perform a reversed case.

Our base case involves proving $M(L + 1)$ where L is the length of the longest element in A . Since there are no length of $\geq L + 1$, this case holds vacuously.

Now, we move to the inductive step. We assume that $M(i + 1)$ is true, so every string s in A is sorted from $s[i + 1]$ onwards, so long as the string has at least $i + 1$ characters. We will attempt to prove that if $M(i + 1)$ is true, the next iteration of the algorithm would make $M(i)$ true. Within A , we consider only the elements with a number of characters $\geq i$. Let us consider any string x that appears before y . Both x and y have at least i characters. There are three cases from here:

- (1) $x[i] < y[i]$. Therefore, these elements are already in sorted order, so these two elements remain in their relative order.

- (2) $x[i] = y[i]$. Since string $x[i+1]$ until the end of $x \leq y[i+1]$ until the end of y , these elements would be added to the value corresponding to key $x[i]$ in sequential order. Therefore, when they are pulled out of the dictionary containing the buckets, they remain in sorted order from character i onwards. This is what ensures the stability of the sort.
- (3) $x[i] > y[i]$. This is the one case where they must be swapped, to reflect how y comes first alphabetically. This process would repeat until either $y[i]$ moves to the front of the list or whichever element s in A to the direct left of y has character $s[i]$ such that $s[i] \leq y[i]$. This is now the same position as the first two cases, so no further movement of y must be made.

Thus, I that have proven if $M(i+1)$ executes and works properly and $M(i)$ then executes, $M(i)$ must also hold true. Therefore, in proving the algorithm is correct in the base case and inductive hypothesis, the algorithm is proven correct.

Run Time Proof: First, we will prove that the counting sort we perform over any the i th character within a a list of strings $O(S_i)$ time complexity, where S_i is the number of strings that have at least that i characters. This sort works by creating a dictionary with sorted keys 'a' to 'z', with its value being a list of the strings for which the i th character of the string is equivalent to the key. Since the dictionary has a maximum length of 26 (1 key-value pair for every letter of the alphabet), the addition of an element has a constant time $O(1)$. The same goes for the reforming of the list using the values in the dictionary, since they are pulled 1 at a time from a dictionary with a constant number of keys. Since this is repeated S_i times, this algorithm has a time complexity of $O(S_i)$.

Now, we can move on to the sequential application of this algorithm to the described type of list. Let us define S_i as the number of elements of Array A that have i characters. Then, by our process, we first order the words of length L by their last character. By the above proof, the ordering of the characters by their L th digit would take $O(S_L)$ time. This process is repeated for words with at least $L-1$ characters, and then for $L-2$ characters, and so on. Since for a string to have i characters, it must also have $i-1$ characters, we can express this as the following:

$$T(n) = O(S_L) + O(S_L + S_{L-1}) + \cdots + O(S_L + S_{L-1} + \cdots + S_1)$$

Simplifying this, we get:

$$= O(L \cdot S_L + (L-1) \cdot S_{L-1} + \cdots + 1 \cdot S_1) = O(n)$$

Thus, this algorithm has a time complexity of $O(n)$